



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 02

NOMBRE COMPLETO: Alejandro Santos Jiménez

N° de Cuenta: 320325586

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 26 de febrero de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

El primer ejercicio consiste en dibujar las iniciales de sus nombres en diagonal de abajo hacia arriba, cada letra de un color diferente

1. Para resolver este problema seguí la lógica del ejercicio de clase ya que solo era poner los vértices de la practica anterior y agregarle la parte de RGB me apoye del ejemplo que venia en la practica.

```
void CrearLetrasyFiguras()
{
    GLfloat vertices_letras[] = {
        //X      Y      Z      R      G      B
        -1.0f,  -1.0f,   0.5f,   0.0f,  0.0f,  1.0f,
        1.0f,   -1.0f,   0.5f,   0.0f,  0.0f,  1.0f,
        1.0f,    1.0f,   0.5f,   0.0f,  0.0f,  1.0f,
        /*1.0f,  1.0f,   0.5f,   1.0f,  0.0f,  0.0f,
        -1.0f,  1.0f,   0.5f,   1.0f,  0.0f,  0.0f,
        -1.0f,  -1.0f,   0.5f,   1.0f,  0.0f,  0.0f,*/

    };
    MeshColor* letras = new MeshColor();
    letras->CreateMeshColor(vertices_letras, 18);
    meshColorList.push_back(letras);
}
```

Para la letra A:

```
GLfloat vertices_letraA[] = {
    // X      Y      Z      R      G      B
    // izquierda
    -0.90f, -0.65f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.85f, -0.55f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.80f, -0.65f, 0.0f, 1.0f, 0.0f, 0.0f,

    -0.75f, -0.55f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.85f, -0.55f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.80f, -0.65f, 0.0f, 1.0f, 0.0f, 0.0f,

    -0.75f, -0.55f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.85f, -0.55f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.80f, -0.45f, 0.0f, 1.0f, 0.0f, 0.0f,

    -0.75f, -0.55f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.70f, -0.45f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.80f, -0.45f, 0.0f, 1.0f, 0.0f, 0.0f,

    -0.75f, -0.35f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.70f, -0.45f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.80f, -0.45f, 0.0f, 1.0f, 0.0f, 0.0f,

    -0.75f, -0.35f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.70f, -0.45f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.65f, -0.35f, 0.0f, 1.0f, 0.0f, 0.0f,

    // parte intermedia
    -0.75f, -0.55f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.70f, -0.45f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.65f, -0.55f, 0.0f, 1.0f, 0.0f, 0.0f,

    -0.60f, -0.45f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.70f, -0.45f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.65f, -0.55f, 0.0f, 1.0f, 0.0f, 0.0f,

    -0.60f, -0.45f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.55f, -0.55f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.65f, -0.55f, 0.0f, 1.0f, 0.0f, 0.0f,

    // parte derecha
    -0.60f, -0.45f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.50f, -0.45f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.55f, -0.55f, 0.0f, 1.0f, 0.0f, 0.0f,

    -0.60f, -0.45f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.50f, -0.45f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.55f, -0.35f, 0.0f, 1.0f, 0.0f, 0.0f,

    -0.60f, -0.45f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.65f, -0.35f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.55f, -0.35f, 0.0f, 1.0f, 0.0f, 0.0f,

    -0.45f, -0.55f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.50f, -0.45f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.55f, -0.55f, 0.0f, 1.0f, 0.0f, 0.0f,

    -0.45f, -0.55f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.50f, -0.65f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.55f, -0.55f, 0.0f, 1.0f, 0.0f, 0.0f,

    -0.45f, -0.55f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.50f, -0.65f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.40f, -0.65f, 0.0f, 1.0f, 0.0f, 0.0f,

    // parte arriba
    -0.75f, -0.35f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.55f, -0.35f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.65f, -0.15f, 0.0f, 1.0f, 0.0f, 0.0f,

    MeshColor* letraA = new MeshColor();
    letraA->CreateMeshColor(vertices_letraA, 288);
    meshColorList.push_back(letraA);
};
```

Para la letra S

```
GLfloat vertices_letraS[] = {
    //      Y      Z      R      G      B
    // parte superior
    -0.20f, 0.15f, 0.0f, 0.0f, 1.0f, 1.0f,
    0.20f, 0.15f, 0.0f, 0.0f, 1.0f, 1.0f,
    -0.20f, 0.25f, 0.0f, 0.0f, 1.0f, 1.0f,

    0.20f, 0.15f, 0.0f, 0.0f, 1.0f, 1.0f,
    0.20f, 0.25f, 0.0f, 0.0f, 1.0f, 1.0f,
    -0.20f, 0.25f, 0.0f, 0.0f, 1.0f, 1.0f,

    // parte izquierda arriba
    -0.20f, 0.05f, 0.0f, 0.0f, 1.0f, 1.0f,
    -0.10f, 0.05f, 0.0f, 0.0f, 1.0f, 1.0f,
    -0.20f, 0.15f, 0.0f, 0.0f, 1.0f, 1.0f,

    -0.10f, 0.05f, 0.0f, 0.0f, 1.0f, 1.0f,
    -0.10f, 0.15f, 0.0f, 0.0f, 1.0f, 1.0f,
    -0.20f, 0.15f, 0.0f, 0.0f, 1.0f, 1.0f,

    // parte central
    -0.20f, -0.05f, 0.0f, 0.0f, 1.0f, 1.0f,
    0.20f, -0.05f, 0.0f, 0.0f, 1.0f, 1.0f,
    -0.20f, 0.05f, 0.0f, 0.0f, 1.0f, 1.0f,

    0.20f, -0.05f, 0.0f, 0.0f, 1.0f, 1.0f,
    0.20f, 0.05f, 0.0f, 0.0f, 1.0f, 1.0f,
    -0.20f, 0.05f, 0.0f, 0.0f, 1.0f, 1.0f,

    // parte derecha abajo
    0.10f, -0.15f, 0.0f, 0.0f, 1.0f, 1.0f,
    0.20f, -0.15f, 0.0f, 0.0f, 1.0f, 1.0f,
    0.10f, -0.05f, 0.0f, 0.0f, 1.0f, 1.0f,

    0.20f, -0.15f, 0.0f, 0.0f, 1.0f, 1.0f,
    0.20f, -0.05f, 0.0f, 0.0f, 1.0f, 1.0f,
    0.10f, -0.05f, 0.0f, 0.0f, 1.0f, 1.0f,

    // parte inferior
    -0.20f, -0.25f, 0.0f, 0.0f, 1.0f, 1.0f,
    0.20f, -0.25f, 0.0f, 0.0f, 1.0f, 1.0f,
    -0.20f, -0.15f, 0.0f, 0.0f, 1.0f, 1.0f,

    0.20f, -0.25f, 0.0f, 0.0f, 1.0f, 1.0f,
    0.20f, -0.15f, 0.0f, 0.0f, 1.0f, 1.0f,
    -0.20f, -0.15f, 0.0f, 0.0f, 1.0f, 1.0f
};

MeshColor* letraS = new MeshColor();
letraS->CreateMeshColor(vertices_letraS, 288);
meshColorList.push_back(letraS);
```

Para la letra J

```
GLfloat vertices_letraJ[] = {
    // X      Y      Z      R      G      B
    // arriba
    0.40f, 0.55f, 0.0f, 0.8f, 0.6f, 1.0f,
    0.80f, 0.55f, 0.0f, 0.8f, 0.6f, 1.0f,
    0.40f, 0.65f, 0.0f, 0.8f, 0.6f, 1.0f,

    0.80f, 0.55f, 0.0f, 0.8f, 0.6f, 1.0f,
    0.80f, 0.65f, 0.0f, 0.8f, 0.6f, 1.0f,
    0.40f, 0.65f, 0.0f, 0.8f, 0.6f, 1.0f,

    // rectangulo derecho
    0.60f, 0.25f, 0.0f, 0.8f, 0.6f, 1.0f,
    0.70f, 0.25f, 0.0f, 0.8f, 0.6f, 1.0f,
    0.60f, 0.55f, 0.0f, 0.8f, 0.6f, 1.0f,

    0.70f, 0.25f, 0.0f, 0.8f, 0.6f, 1.0f,
    0.70f, 0.55f, 0.0f, 0.8f, 0.6f, 1.0f,
    0.60f, 0.55f, 0.0f, 0.8f, 0.6f, 1.0f,

    // rectangulo abajo
    0.40f, 0.15f, 0.0f, 0.8f, 0.6f, 1.0f,
    0.70f, 0.15f, 0.0f, 0.8f, 0.6f, 1.0f,
    0.40f, 0.25f, 0.0f, 0.8f, 0.6f, 1.0f,

    0.70f, 0.15f, 0.0f, 0.8f, 0.6f, 1.0f,
    0.70f, 0.25f, 0.0f, 0.8f, 0.6f, 1.0f,
    0.40f, 0.25f, 0.0f, 0.8f, 0.6f, 1.0f,

    // punta inferior izquierda
    0.40f, 0.25f, 0.0f, 0.8f, 0.6f, 1.0f,
    0.50f, 0.25f, 0.0f, 0.8f, 0.6f, 1.0f,
    0.40f, 0.35f, 0.0f, 0.8f, 0.6f, 1.0f,

    0.50f, 0.25f, 0.0f, 0.8f, 0.6f, 1.0f,
    0.50f, 0.35f, 0.0f, 0.8f, 0.6f, 1.0f,
    0.40f, 0.35f, 0.0f, 0.8f, 0.6f, 1.0f
};

MeshColor* letraJ = new MeshColor();
letraJ->CreateMeshColor(vertices_letraJ, 144);
meshColorList.push_back(letraJ);
```

Ya en ejecución :

```
//Letra A
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

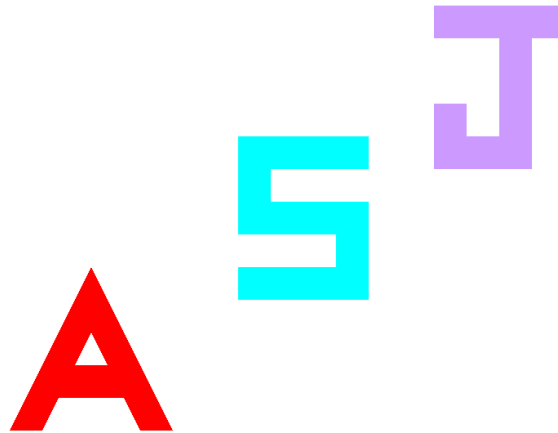
meshColorList[1] -> RenderMeshColor();

//Letra S
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

meshColorList[2] -> RenderMeshColor();
//Letra j
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

meshColorList[3] -> RenderMeshColor();
```

Practica 2: Proyecciones, transformaciones



2.- Generar el dibujo de la casa de la clase, pero en lugar de instanciar triángulos y cuadrados será instanciando pirámides y cubos, para esto se requiere crear shaders diferentes de los colores: rojo, verde, azul, café y verde oscuro en lugar de usar el shader con el color clamp. Se debe de entregar los archivos de shader diferentes dentro de un zip adicional al main y documento escrito.

Para esta practica se crearon 5 shaders de color

```
//shaders nuevos se crearían acá
static const char* AzulShader = "shaders/azul.frag";
static const char* CafeShader = "shaders/cafe.frag";
static const char* RojoShaderColor = "shaders/rojo.frag";
static const char* VerdeShaderColor = "shaders/verde.frag";
static const char* VerdeOShaderColor = "shaders/verdeosc.frag";
```

```
void CreateShaders()
{
    Shader *shader1 = new Shader(); //shader para usar índices: cubo y pirámide índice 0
    shader1->CreateFromFiles(vShader, fShader);
    shaderList.push_back(*shader1);

    Shader *shader2 = new Shader(); //shader para usar color como parte del VAO: letras índice 1
    shader2->CreateFromFiles(vShaderColor, fShaderColor);
    shaderList.push_back(*shader2);

    //shader color para la casa APARTIR DEL INDICE 2

    Shader* shaderRojo = new Shader();
    shaderRojo->CreateFromFiles(vShader, RojoShaderColor);
    shaderList.push_back(*shaderRojo);

    Shader* shaderAzul = new Shader();
    shaderAzul->CreateFromFiles(vShader, AzulShader);
    shaderList.push_back(*shaderAzul);

    Shader* shaderCafe = new Shader();
    shaderCafe->CreateFromFiles(vShader, CafeShader);
    shaderList.push_back(*shaderCafe);

    Shader* shaderVerdeOsc = new Shader();
    shaderVerdeOsc->CreateFromFiles(vShader, VerdeOShaderColor);
    shaderList.push_back(*shaderVerdeOsc);

    Shader* shaderVerde = new Shader();
    shaderVerde->CreateFromFiles(vShader, VerdeShaderColor);
    shaderList.push_back(*shaderVerde);
}
```

Se comento el ejercicio uno para que no hubiera problemas, la implementación cambio, ya que ahora jugaba la coordenada “z” quedando de la siguiente manera:

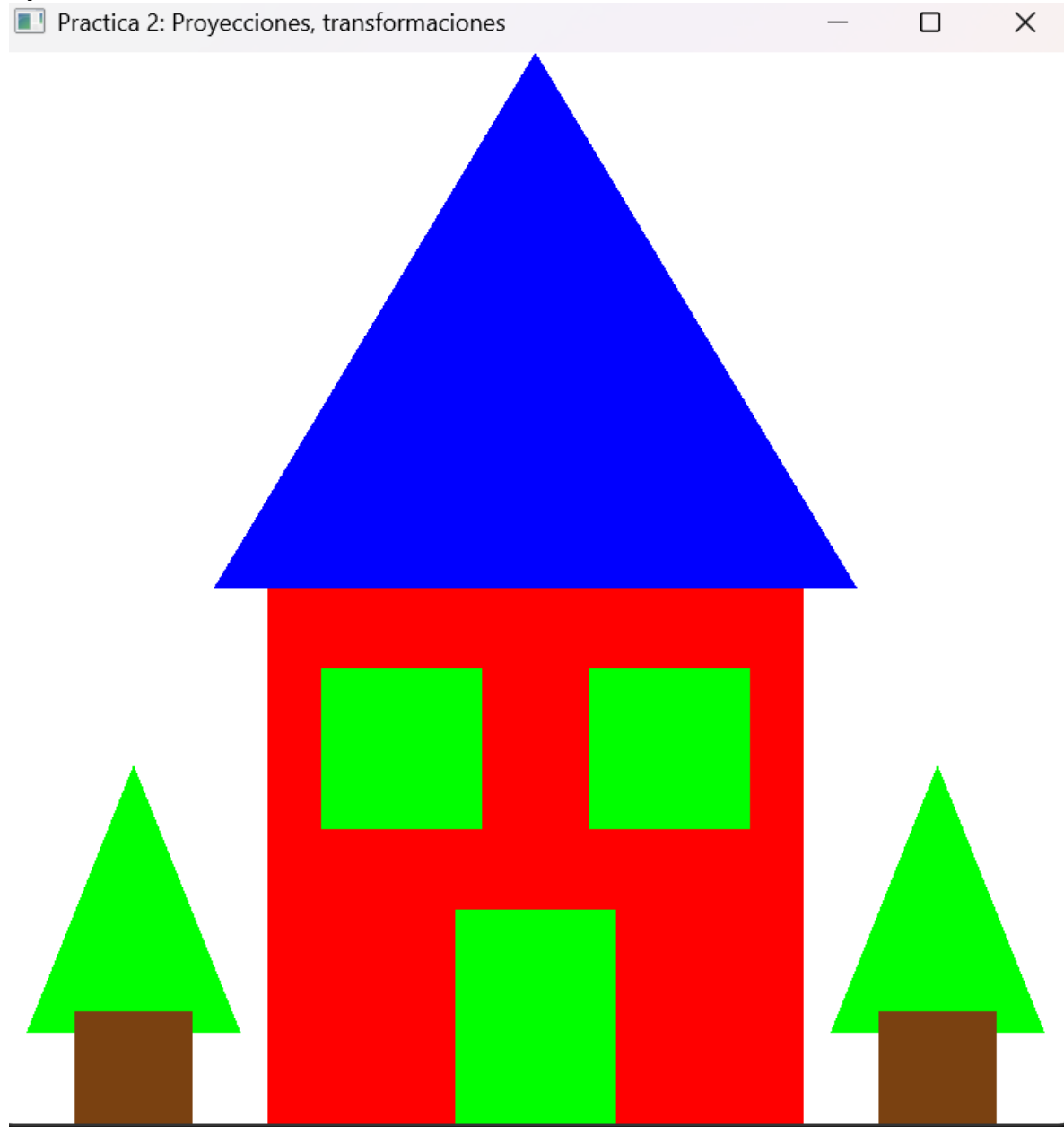
```
//Parte 2 practica 2
//Para el cubo y la pirámide se usa el primer set de shaders con índice 0 en ShaderList
// cubo central casa
shaderList[2].useShader();
uniformModel = shaderList[2].getModelLocation();
uniformProjection = shaderList[2].getProjectLocation();

//Inicializar matriz de dimensión 4x4 que servirá como matriz de modelo para almacenar las transformaciones geomé
//Em
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::translate(model, glm::vec3(0.0f, -0.5f, -0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[1] -> RenderMesh();

shaderList[3].useShader();
uniformModel = shaderList[3].getModelLocation();
uniformProjection = shaderList[3].getProjectLocation();
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.5f, -3.0f));
model = glm::scale(model, glm::vec3(1.2f, 1.0f, 1.2f)); // Un techo más ancho
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[0] -> RenderMesh();

shaderList[4].useShader();
uniformModel = shaderList[4].getModelLocation();
uniformProjection = shaderList[4].getProjectLocation();
// Tronco Derecho
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.75f, -0.9f, -3.0f));
model = glm::scale(model, glm::vec3(0.22f, 0.22f, 0.22f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
```

Ejecución:



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

No se tuvo problemas con la practica, se logro el objetivo.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.

La mayor complejidad fue pasar de 2D a 3D fue crear los nuevos archivos de shaders, ya que estas figuras se colorean de distinta forma

- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

No faltó explicar nada, todo estuvo claro.

- c. Conclusión

Esta practica no estaba tan compleja ya que tenia que ver con la pasada, entonces eso facilitó el entendimiento de los nuevos conceptos.

1. Bibliografía en formato APA